

An Example of Topic Modeling

Giulio Beltramin

2026-10-09

Topic Modelling with Multinomial - Dirichlet Model

In this notebook, we see an example of Topic Modelling using the Multinomial - Dirichlet model. We recall that this model is defined as follows:

Let X_i be the word i in the document we are analyzing, for $i = 1, \dots, n$. The law for X_i is the following:

$$\mathbb{P}(X_i = j) = \lambda_j \quad j = 1, \dots, |V|,$$

where $|V|$ is the size of the vocabulary and j is a label for the word.

Of course, in a document words will have repetitions. Since the order in which a word appear is not important, we can define Y_j being the frequency of word j in the document.

Bayesian Model

$$(y_1, \dots, y_n) \mid \boldsymbol{\lambda} \stackrel{iid}{\sim} \text{Multinomial}(\boldsymbol{\lambda}); \\ \boldsymbol{\lambda} \sim \text{Dirichlet}(\boldsymbol{\alpha}).$$

We recall that:

$$f(\mathbf{y} \mid \boldsymbol{\lambda}) = \frac{n!}{y_1! \dots y_{|V|}!} \prod_{j=1}^{|V|} \lambda_j^{y_j}, \quad \pi(\boldsymbol{\lambda}) = \frac{1}{B(\boldsymbol{\alpha})} \prod_{j=1}^{|V|} \lambda_j^{\alpha_j - 1},$$

with $y_j \in \mathbb{N} \cup \{0\}$ for all j , $\sum_{j=1}^{|V|} y_j = n$, $\alpha_j \geq 0$ for all j and $\sum_{j=1}^{|V|} \alpha_j = 1$.

```
# Load required libraries
library(httr2)
library(jsonlite)
library(tidytext)
```

```
## Warning: il pacchetto 'tidytext' è stato creato con R versione 4.4.3
```

```
library(stopwords)
```

```
## Warning: il pacchetto 'stopwords' è stato creato con R versione 4.4.3
```

```
library(dplyr)
```

```
##  
## Caricamento pacchetto: 'dplyr'  
  
## I seguenti oggetti sono mascherati da 'package:stats':  
##  
##     filter, lag  
  
## I seguenti oggetti sono mascherati da 'package:base':  
##  
##     intersect, setdiff, setequal, union
```

```
library(ggplot2)
```

```
# Query Wikipedia API for Roberto Baggio's page extract  
url <- "https://en.wikipedia.org/w/api.php"
```

```
req <- request(url) %>%  
  req_headers("User-Agent" = "Mozilla/5.0") %>%  
  req_url_query(  
    action = "query",  
    format = "json",  
    titles = "Roberto_Baggio",  
    prop = "extracts",  
    explaintext = "1"  
  )
```

```
response <- req_perform(req)  
data <- resp_body_json(response)
```

```
# Extract raw text from the response structure  
page_id <- names(data$query$pages)[[1]]  
text <- data$query$pages[[page_id]]$extract
```

```
# turn the text into token and keep only the words, eliminating punctuation  
tokens <- unlist(strsplit(tolower(text), "[^a-z]+"))  
tokens <- tokens[tokens != ""]
```

```
# eliminate english stopwords: words commonly used that are not useful to assess the meaning of a text,  
stopw <- stopwords::stopwords("en")
```

```
tokens <- tokens[!tokens %in% stopw]
```

Let us print the 10 most probable words...

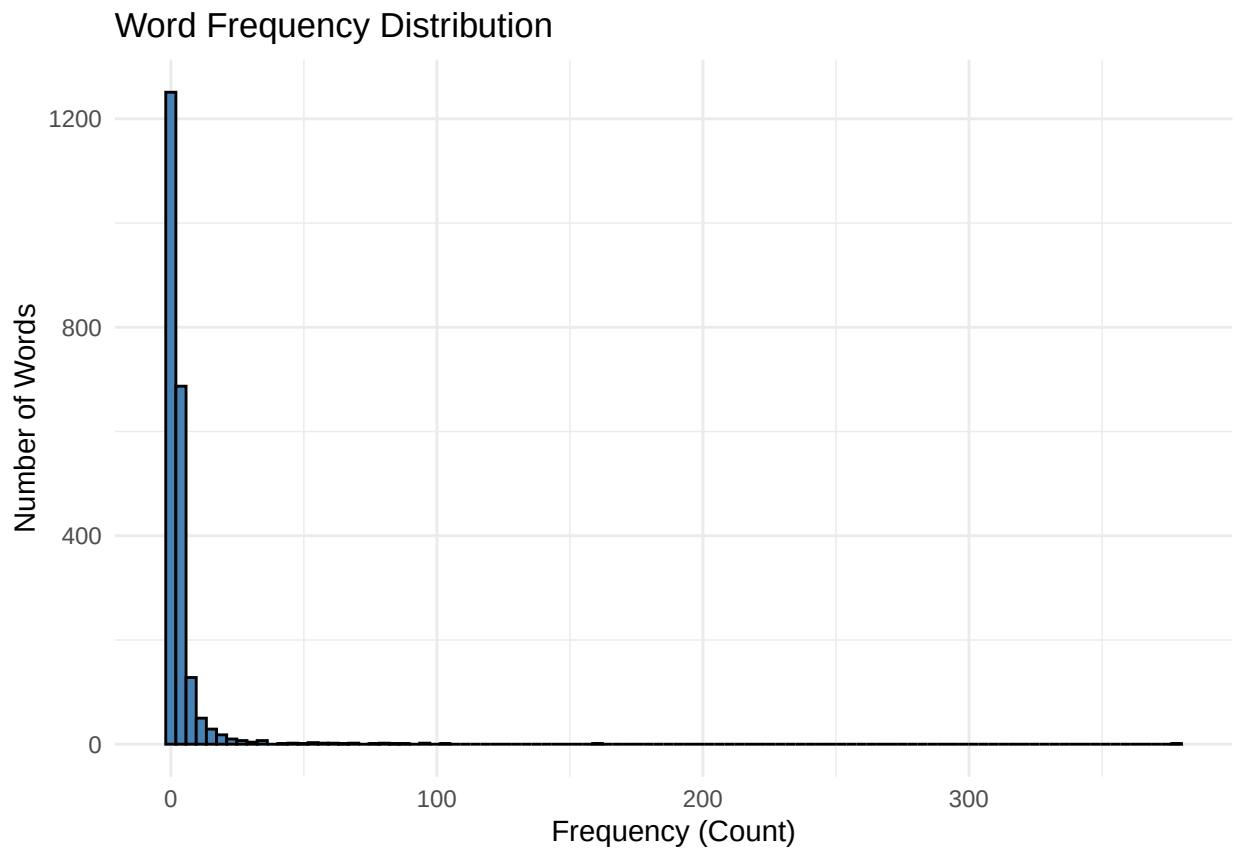
```
# Count frequencies and get top 10 most common words  
counts <- sort(table(tokens), decreasing = TRUE)  
head(counts, 10)
```

```
## tokens  
## baggio      s world  cup  italy serie goals goal scored also  
##    379    162   104   96   95    86   85    79    79    75
```

... and **always** remember to plot your data!

```
df_counts <- as.data.frame(counts)
colnames(df_counts) <- c("word", "count")

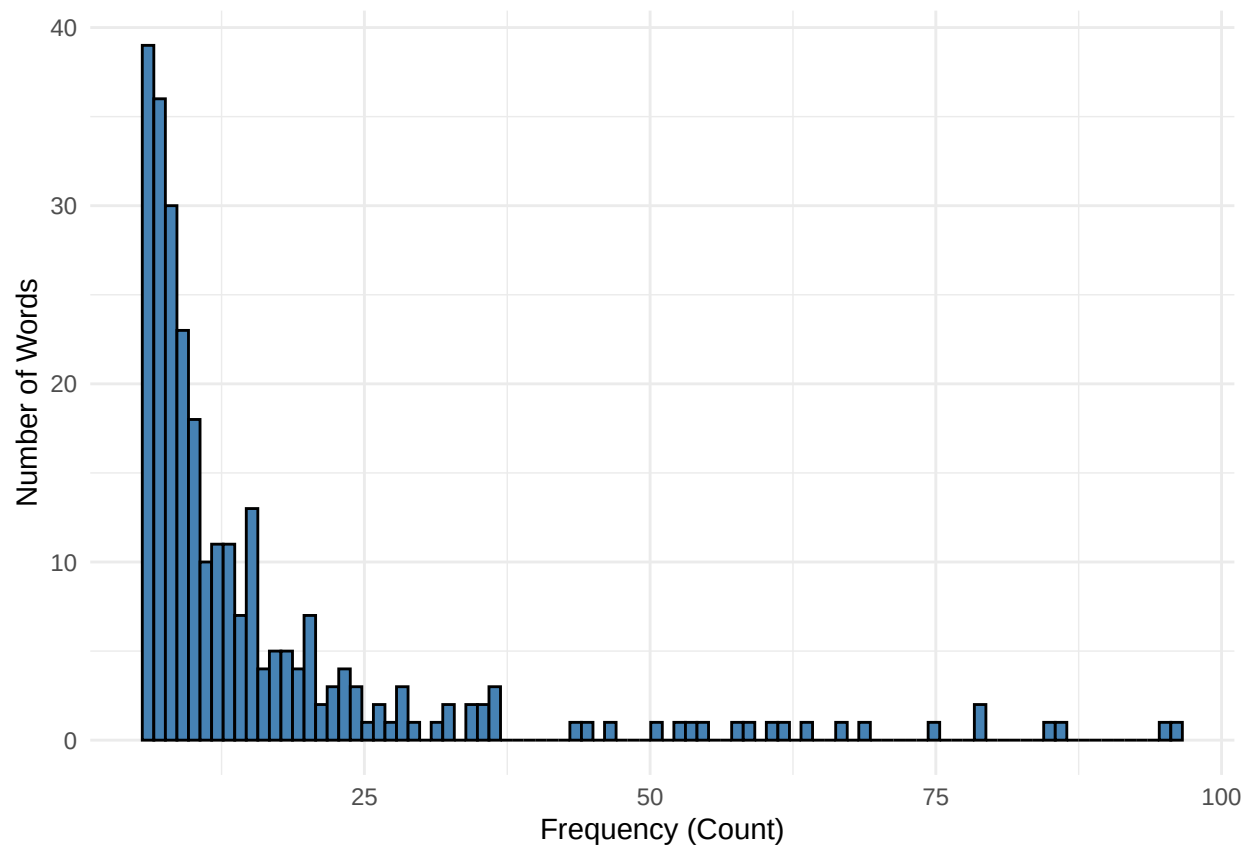
ggplot(df_counts, aes(x = count)) +
  geom_histogram(bins = 100, fill = "steelblue", color = "black") +
  theme_minimal() +
  labs(
    title = "Word Frequency Distribution",
    x = "Frequency (Count)",
    y = "Number of Words"
  )
)
```



```
# Let's keep the words with frequency between 6 and 99
counts_filtered <- counts[counts > 5 & counts < 100]

df_filtered <- data.frame(count = as.numeric(counts_filtered))

ggplot(df_filtered, aes(x = count)) +
  geom_histogram(bins = 90, fill = "steelblue", color = "black") +
  theme_minimal() +
  labs(
    x = "Frequency (Count)",
    y = "Number of Words"
  )
)
```



Bayesian analysis

Posterior Distribution

We saw in the previous TA that the posterior distribution is:

$$\pi(\boldsymbol{\lambda} \mid \mathbf{y}) \sim \text{Dirichlet}(\boldsymbol{\alpha} + \mathbf{y})$$

```
# Extract vocabulary (token names) and calculate its length
vocabulary <- names(counts_filtered)
len_v <- length(vocabulary)

# Frequency counts array
y <- as.numeric(counts_filtered)

# Symmetric Dirichlet prior
alpha <- rep(1, len_v)
```

Predictive Distribution

$$p(x_{n+1} = j \mid x_1, \dots, x_n) = \frac{\alpha_j + y_j}{n + \sum_{h=1}^{|V|} \alpha_h}$$

for every $j = 1, \dots, |V|$

```

# Vectorized implementation
pred_dist <- (alpha + y) / (sum(alpha) + sum(y))

# Get the 5 most probable words
ind <- order(pred_dist, decreasing = TRUE)[1:5]
for (i in ind) {
  cat(sprintf("word: %s, prob: %s\n", vocabulary[i], pred_dist[i]))
}

## word: cup, prob: 0.0204944010141559
## word: italy, prob: 0.0202831185294739
## word: serie, prob: 0.0183815761673357
## word: goals, prob: 0.0181702936826537
## word: goal, prob: 0.0169025987745616

# Get the 5 least probable words
ind <- order(pred_dist, decreasing = FALSE)[1:5]
for (i in ind) {
  cat(sprintf("word: %s, prob: %s\n", vocabulary[i], pred_dist[i]))
}

## word: another, prob: 0.00147897739277414
## word: argentina, prob: 0.00147897739277414
## word: arrigo, prob: 0.00147897739277414
## word: brazil, prob: 0.00147897739277414
## word: christian, prob: 0.00147897739277414

```

Test randomness

$$H_0 : \lambda = \left(\frac{1}{|V|} \cdots \frac{1}{|V|} \right) \quad \text{vs.} \quad H_0 : \lambda \neq \left(\frac{1}{|V|} \cdots \frac{1}{|V|} \right)$$

Let $\lambda_0 = \left(\frac{1}{|V|} \cdots \frac{1}{|V|} \right)$, then

$$BF_{01} = \frac{L(\mathbf{y}|\lambda_0)}{\int_{S^{|V|}} L(\mathbf{y}|\lambda) \pi(\lambda) d\lambda} = \frac{L(\mathbf{y}|\lambda_0)}{m(\mathbf{y})}$$

We saw in the previous lab that $m(\mathbf{y}) = \frac{n!}{y_1! \cdots y_{|V|}!} \cdot \frac{B(\boldsymbol{\alpha} + \mathbf{y})}{B(\boldsymbol{\alpha})}$. We can therefore see that

$$BF_{01} = \frac{\prod_{j=1}^{|V|} \left(\frac{1}{|V|} \right)^{y_j}}{\frac{B(\boldsymbol{\alpha} + \mathbf{y})}{B(\boldsymbol{\alpha})}}.$$

```

multi_beta_fn <- function(alpha) {
  prod(gamma(alpha)) / gamma(sum(alpha))
}

lambda0 <- rep(1 / len_v, len_v)

num <- prod(lambda0 ^ y)
den <- multi_beta_fn(alpha + y) / multi_beta_fn(alpha)

print(log10(num / den))

```

```
## [1] NaN
```

R returns a NaN (not a number). This (almost) always happens when we try to compute products of terms involving factorials.

What to do? Move computations to log scale! In this case:

```
# Define log10 gamma function
log10gamma <- function(x) {
  lgamma(x) / log(10.0)
}

# Define log10 multivariate beta function
log_multi_beta_fn <- function(alpha) {
  sum(log10gamma(alpha)) - log10gamma(sum(alpha))
}

log_num <- sum(y * log10(lambda0))
log_den <- log_multi_beta_fn(alpha + y) - log_multi_beta_fn(alpha)

print(log_num - log_den)
```

```
## [1] -459.0607
```